



本教學投影片係屬教科書著作之延伸，亦受著作權法之保護。

請依出版社授權使用規範利用本教學資源，非經授權許可不得以影印、拷貝、列印等方法進行重製，亦不得改作、公開展示著作內容，亦請勿對第三人公開傳輸、散布。



Chapter 2

Boolean Algebra and Logic Gates

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Objectives (1/2)

- Gain a basic understanding of postulates used to form algebraic structures.
- Understand the Huntington Postulates.
- Understand the basic theorems and postulates of Boolean algebra.
- Know how to develop a logic diagram from a Boolean function; Know how to derive a Boolean function from a logic diagram.
- Know how to apply DeMorgan's theorems.
- Know how to express a Boolean function as a truth table; know how to derive a Boolean function from a truth table.

Objectives (2/2)

- Know how to express a Boolean function as a sum of minterms and as a product of maxterms.
- Be able to convert from a sum of minterms to a product of maxterms, and vice versa.
- Be able to form a two-level gate structure from a Boolean function in sum of products form; know how to form a two-level gate structure from a Boolean function in product of sums form.
- Be able to implement a Boolean function with NAND and inverter gates; know how to implement a Boolean function with NOR and inverter gates.

2.2 Basic Definitions (1/4)

- Axiom (公理)
 - Formulas that are satisfied by every assignment of values.
 - 「就是這樣，不要問為什麼」、「不證自明」
- Postulate (公設、假說)
 - Theory-specific assumptions.
 - 「曾經有爭議的特定理論但目前大部分專家有共識後將其充當基本假設的一種公式」
- A *binary operator* defined on a set S of elements is a rule that assigns, to each pair of elements from S , a unique element from S .

2.2 Basic Definitions (2/4)

- The most common postulates used to formulate various algebraic structures:
 1. **Closure (封閉性).** A set S is closed with respect to a binary operator if, for every pair of elements of S , the binary operator specifies a rule for obtaining a unique element of S .
 2. **Associative law (結合律).** A binary operator $*$ on a set S is said to be associative whenever

$$(x*y)*z = x*(y*z) \text{ for all } x, y, z, \in S$$
 3. **Commutative law (交換律).** A binary operator $*$ on a set S is said to be commutative whenever

$$x*y = y*x \text{ for all } x, y \in S$$
 4. **Identity element (單位元素).** A set S is said to have an identity element with respect to a binary operation $*$ on S if there exists an element $e \in S$ with the property that

$$e*x = x*e = x \text{ for every } x \in S$$

2.2 Basic Definitions (3/4)

Example: The element 0 is an identity element with respect to the binary operator $+$ on the set of integers $I = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$, since $x + 0 = 0 + x = x$ for any $x \in I$. The set of natural numbers, N , has no identity element since 0 is excluded from the set.

5. **Inverse (反元素).** A set S having the identity element e with respect to a binary operator $*$ is said to have an inverse if, for every $x \in S$, there exists an element $y \in S$ such that

$$x * y = e$$

Example: In the set of integers, I , and the operator $+$, with $e = 0$, the inverse of an element a is $(-a)$, since $a + (-a) = 0$.

6. **Distributive law (分配律).** If $*$ and $\cdot$ are two binary operators on a set S , $*$ is said to be distributive over $\cdot$ whenever

$$x * (y \cdot z) = (x * y) \cdot (x * z)$$

2.2 Basic Definitions (4/4)

- A *field* (域或場) is an example of an algebraic structure.
 - The *field* of real numbers is the basis for arithmetic and ordinary algebra.
 - The binary operator $+$ defines addition.
 - The additive identity is 0.
 - The additive inverse defines subtraction.
 - The binary operator $\cdot$ defines multiplication.
 - The multiplicative identity is 1.
 - For $a \neq 0$, $1/a$ is the multiplicative inverse of a that also defines division (i.e., $a \cdot 1/a = 1$).
 - The only distributive law applicable is that of $\cdot$ over $+$:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

2.3 Axiomatic Definition of Boolean Algebra (1/6)

- In 1854, George Boole developed an algebraic system now called *Boolean algebra*.
- Boolean algebra is an algebraic structure defined by a set of elements, B , together with two binary operators, “+” and “ $\cdot$ ”, provided that the following (Huntington) postulates are satisfied:
 1. (a) The structure is closed with respect to the operator “+”.
 - (b) The structure is closed with respect to the operator “ $\cdot$ ”.
 2. (a) The element 0 is an identity element with respect to +; i.e., $x + 0 = 0 + x = x$.
 - (b) The element 1 is an identity element with respect to $\cdot$; i.e., $x \cdot 1 = 1 \cdot x = x$.

2.3 Axiomatic Definition of Boolean Algebra (2/6)

3. (a) The structure is commutative with respect to $+$; i.e., $x + y = y + x$.
 (b) The structure is commutative with respect to $\cdot$; i.e., $x \cdot y = y \cdot x$.
4. (a) The operator $\cdot$ is distributive over $+$; i.e., $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$.
 (b) The operator $+$ is distributive over $\cdot$; i.e., $x + (y \cdot z) = (x + y) \cdot (x + z)$.
5. For every element $x \in B$, there exists an element $x' \in B$ (called the complement of x) such that
 - (a) $x + x' = 1$
 - (b) $x \cdot x' = 0$.
6. There exist at least two elements $x, y \in B$ such that $x \neq y$.

2.3 Axiomatic Definition of Boolean Algebra (3/6)

- Comparing Boolean algebra with arithmetic and ordinary algebra
 1. Huntington postulates do not include the associative law. However, this law holds for Boolean algebra and can be derived (for both operators) from the other postulates.
 2. The distributive law of $+$ over $\cdot$ (i.e., $x + (y \cdot z) = (x + y) \cdot (x + z)$) is valid for Boolean algebra, but not for ordinary algebra.
 3. Boolean algebra does not have additive or multiplicative inverses; therefore, there are no subtraction or division operations.
 4. Postulate 5 defines an operator called the complement that is not available in ordinary algebra.
 5. Ordinary algebra deals with the real numbers, which constitute an infinite set of elements. Boolean algebra deals with the as yet undefined set of elements, B , but in the two-valued Boolean algebra defined next, B is defined as a set with only two elements, 0 and 1.

2.3 Axiomatic Definition of Boolean Algebra (4/6)

- Two-Valued Boolean Algebra, i.e., $B = \{0, 1\}$
 - The rules of operations for $+$ and $\cdot$ are as below.

x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

- Huntington postulates verifications
 - Closure
 - The identity elements: (a) $+$ $\rightarrow$ 0; (b) $\cdot$ $\rightarrow$ 1
 - The commutative laws

2.3 Axiomatic Definition of Boolean Algebra (5/6)

- The distributive laws

x	y	z	$y + z$	$x \cdot (y + z)$	$x \cdot y$	$x \cdot z$	$(x \cdot y) + (x \cdot z)$
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	1	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

- Complement
 - $x + x' = 1$, since $0 + 0' = 0 + 1 = 1$ and $1 + 1' = 1 + 0 = 1$
 - $x \cdot x' = 0$, since $0 \cdot 0' = 0 \cdot 1 = 0$ and $1 \cdot 1' = 1 \cdot 0 = 0$
- Has two distinct elements 1 and 0, with $0 \neq 1$

2.3 Axiomatic Definition of Boolean Algebra (6/6)

- Note
 - The set B is a set of two elements
 - “+” is the OR operator doing the OR operation
 - “ $\cdot$ ” is the AND operator doing the AND operation
 - “ $'$ ” is a complement operator doing NOT operation
 - The “binary logic” in Section 1.9 is, in fact, a two-valued Boolean algebra

2.4 Basic Theorems and Properties of Boolean Algebra (1/7)

• Duality

- The binary operators are interchanged; AND $\leftrightarrow$ OR
- The identity elements are interchanged; 1 $\leftrightarrow$ 0

Table 2.1
Postulates and Theorems of Boolean Algebra

Postulate 2	(a)	$x + 0 = x$	(b)	$x \cdot 1 = x$
Postulate 5	(a)	$x + x' = 1$	(b)	$x \cdot x' = 0$
Theorem 1	(a)	$x + x = x$	(b)	$x \cdot x = x$
Theorem 2	(a)	$x + 1 = 1$	(b)	$x \cdot 0 = 0$
Theorem 3, involution (乘方)		$(x')' = x$		
Postulate 3, commutative	(a)	$x + y = y + x$	(b)	$xy = yx$
Theorem 4, associative	(a)	$x + (y + z) = (x + y) + z$	(b)	$x(yz) = (xy)z$
Postulate 4, distributive	(a)	$x(y + z) = xy + xz$	(b)	$x + yz = (x + y)(x + z)$
Theorem 5, DeMorgan	(a)	$(x + y)' = x'y'$	(b)	$(xy)' = x' + y'$
Theorem 6, absorption	(a)	$x + xy = x$	(b)	$x(x + y) = x$

- Sometimes the operator is omitted if no confusion occurred.

2.4 Basic Theorems and Properties of Boolean Algebra (2/7)

- Proof of Theorem 1(a): $x + x = x$

$$\begin{aligned}
 x + x &= (x + x) \cdot 1 && \text{by postulate: } 2(b) \\
 &= (x + x) (x + x') && 5(a) \\
 &= x + xx' && 4(b) \\
 &= x + 0 && 5(b) \\
 &= x && 2(a)
 \end{aligned}$$

- Proof of Theorem 1(b): $x \cdot x = x$

$$\begin{aligned}
 x \cdot x &= xx + 0 && \text{by postulate: } 2(b) \\
 &= xx + xx' && 5(b) \\
 &= x(x + x') && 4(a) \\
 &= x \cdot 1 && 5(a) \\
 &= x && 2(b)
 \end{aligned}$$

2.4 Basic Theorems and Properties of Boolean Algebra (3/7)

- Proof of Theorem 2

$$\begin{aligned}
 2(a): \quad x + 1 &= 1 \cdot (x + 1) && \text{by postulate: } 2(b) \\
 &= (x + x')(x + 1) && 5(a) \\
 &= x + x' \cdot 1 && 4(b) \\
 &= x + x' && 2(b) \\
 &= 1 && 5(a)
 \end{aligned}$$

$$2(b): x \cdot 0 = 0 \text{ by duality}$$

- Proof of Theorem 3: $(x')' = x$

- Postulate 5 defines the complement of x , $x + x' = 1$ and $x \cdot x' = 0$
- The complement of x' is x is also $(x')'$

2.4 Basic Theorems and Properties of Boolean Algebra (4/7)

- Proof of Theorem 6

$$\begin{aligned}
 6(a) \quad x + xy &= x \cdot 1 + xy \\
 &= x(1 + y) \\
 &= x \cdot 1 \\
 &= x
 \end{aligned}$$

$$6(b) \quad x(x + y) = x \text{ by duality}$$

- Proof the Boolean algebra by means of the truth table

x	y	xy	$x + xy$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

2.4 Basic Theorems and Properties of Boolean Algebra (5/7)

- DeMorgan's Theorems

- $(x+y)' = x'y'$
- $(x \cdot y)' = x' + y'$

x	y	$x+y$	$(x+y)'$	x'	y'	$x'y'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

2.4 Basic Theorems and Properties of Boolean Algebra (6/7)

- Operator Precedence

- Parentheses
- NOT
- AND
- OR

- Examples

- $xy' + z$
- $(xy + z)'$

2.4 Basic Theorems and Properties of Boolean Algebra (7/7)

- **Practice Exercise 2.1** Using the basic theorems and postulates of Boolean algebra, simplify the Boolean expression: $F = x'y'z + xyz + x'yz + xy'z$

Answer: $F = z$

- **Practice Exercise 2.2** Develop a truth table for the Boolean expression: $F = x'y'z$

Hint:

1. a primed variable $\rightarrow$ "0"
2. an unprimed variable $\rightarrow$ "1"

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

2.5 Boolean Functions (1/3)

- A Boolean function
 - binary variables
 - binary operators OR and AND
 - unary operator NOT
 - parentheses
- Examples
 - $F_1 = x + y'z$
 - $F_2 = x'y'z + x'yz + xy'$

- The truth table for F_1 and F_2

Hint:

1. a primed variable $\rightarrow$ “0”
2. an unprimed variable $\rightarrow$ “1”

Table 2.2
Truth Tables for F_1 and F_2

x	y	z	F_1	F_2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	0

2.5 Boolean Functions (2/3)

- Implementation with logic gates for $F_1 = x + y'z$

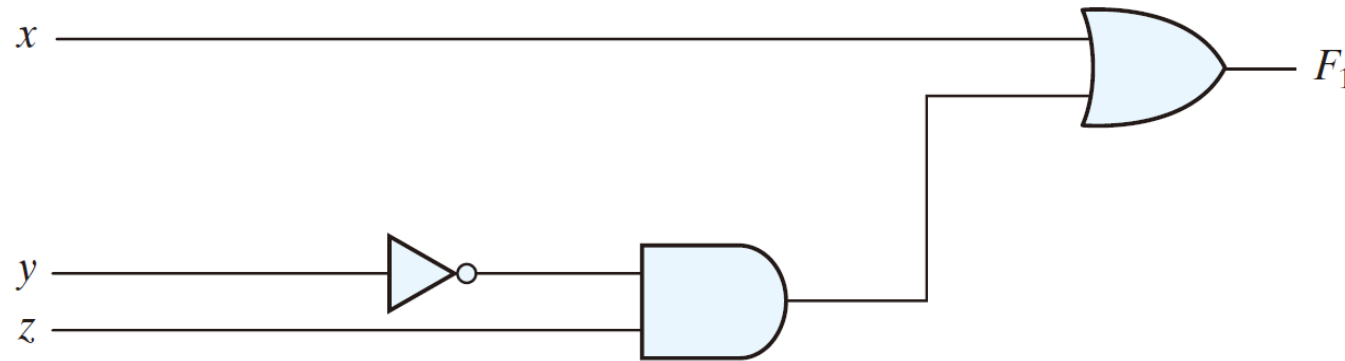


Figure 2.1

Logic diagram for the Boolean function $F_1 = x + y'z$

2.5 Boolean Functions (3/3)

- Implementation with logic gates for $F_2 = x'y'z + x'yz + xy'$

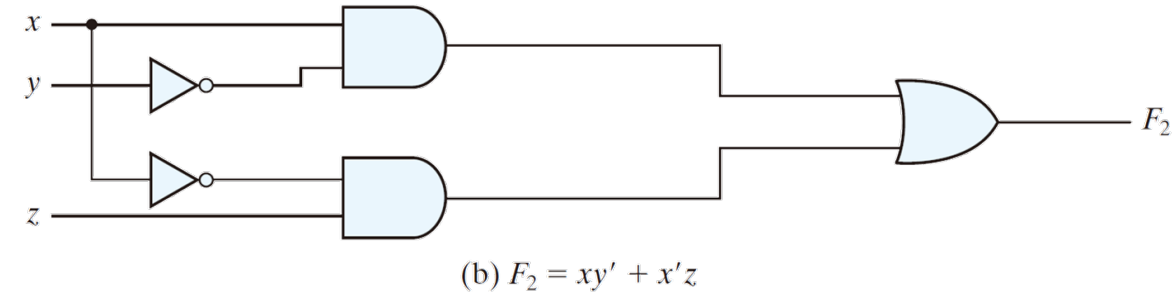
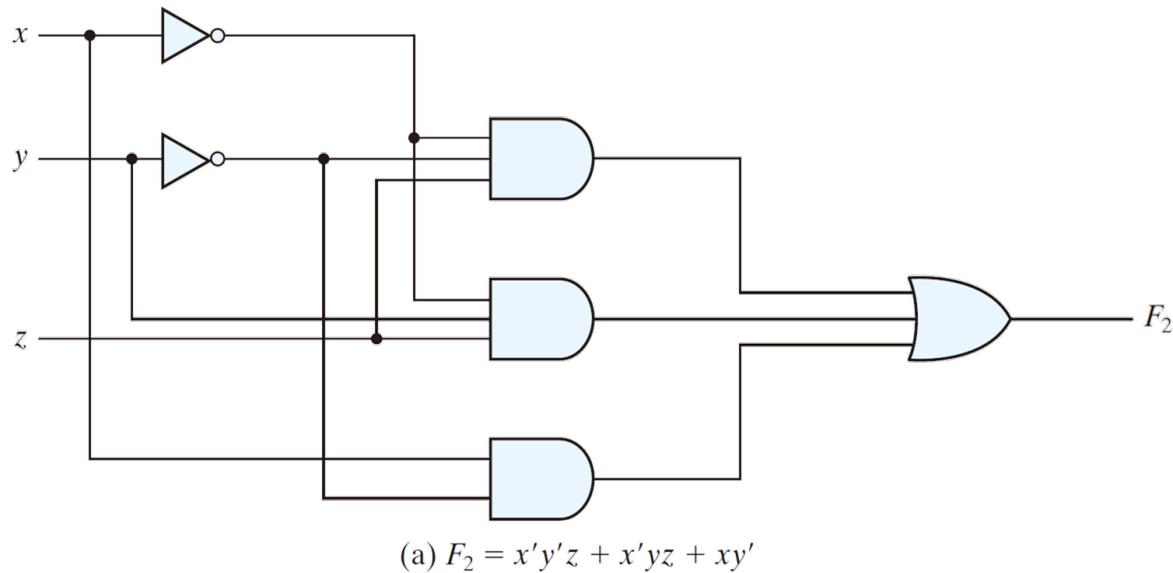


FIGURE 2.2
Implementation of Boolean function F_2 with gates

2.5.1 Algebraic Manipulation (代數演算) (1/2)

- To minimize Boolean expressions
 - term (項)
 - an implementation with a gate
 - e.g., $x'y'z$, $x'yz$, xy'
 - literal (文字字元)
 - a primed or unprimed variable
 - an input to a gate
 - e.g., x , x'
 - The minimization of the number of literals and the number of terms $\Rightarrow$ a circuit with less equipment
 - Figure 2.2 (a): 8 literals and 3 terms
 - Figure 2.2 (b): 4 literals and 2 terms

2.5.1 Algebraic Manipulation (2/2)

- **Example 2.1** Simplify the following Boolean functions to a minimum number of literals
 1. $x(x' + y) = xx' + xy = 0 + xy = xy$.
 2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y$. This is the duality of function 1.
 - 3.1 $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y' + 0) = x$.
 - 3.2 $(x + y)(x + y') = x + y'y = x + 0 = x$
 4. $xy + x'z + yz = xy + x'z + yz(\mathbf{x + x'}) = xy + x'z + xyz + x'yz$
 $= xy(1 + z) + x'z(1 + y) = xy + x'z$.
 5. $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$, by duality from function 4.

2.5.2 Complement of a Function (1/6)

- By using DeMorgan's theorem
 - Three-variable form of the first DeMorgan's theorem
 - $(A + B + C)' = (A + x)'$ let $B + C = x$
 $= A'x'$ by Theorem 5(a) (DeMorgan)
 $= A'(B + C)'$ substitute $B + C = x$
 $= A'(B'C')$ by Theorem 5(a) (DeMorgan)
 $= A'B'C'$ by Theorem 4(b) (associative)
 - Generalizations of the DeMorgan's theorem to any variables
 - $(A + B + C + \dots + K)' = A'B'C' \dots K'$
 - $(ABC \dots K)' = A' + B' + C' + \dots + K'$
- Procedures to obtain the complement of a function F
 - Interchanging AND to OR and OR to AND
 - Taking the complement of each variable in F

2.5.2 Complement of a Function (2/6)

- **Example 2.2** Find the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$. By applying DeMorgan's theorems as many times as necessary, the complements are obtained as follows:

$$F'_1 = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$\begin{aligned} F'_2 &= [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')'(yz)' \\ &= x' + (y + z)(y' + z') = x' + yz' + y'z \end{aligned}$$

2.5.2 Complement of a Function (3/6)

- **Example 2.3** Find the complement of the functions F_1 and F_2 of Example 2.2 by taking their duals and complementing each literal.

1. $F_1 = x'yz' + x'y'z.$

The dual of is $(x' + y + z')(x' + y' + z).$

Complement each literal: $(x + y' + z)(x + y + z') = F'_1.$

2. $F_2 = x(y'z' + yz)$

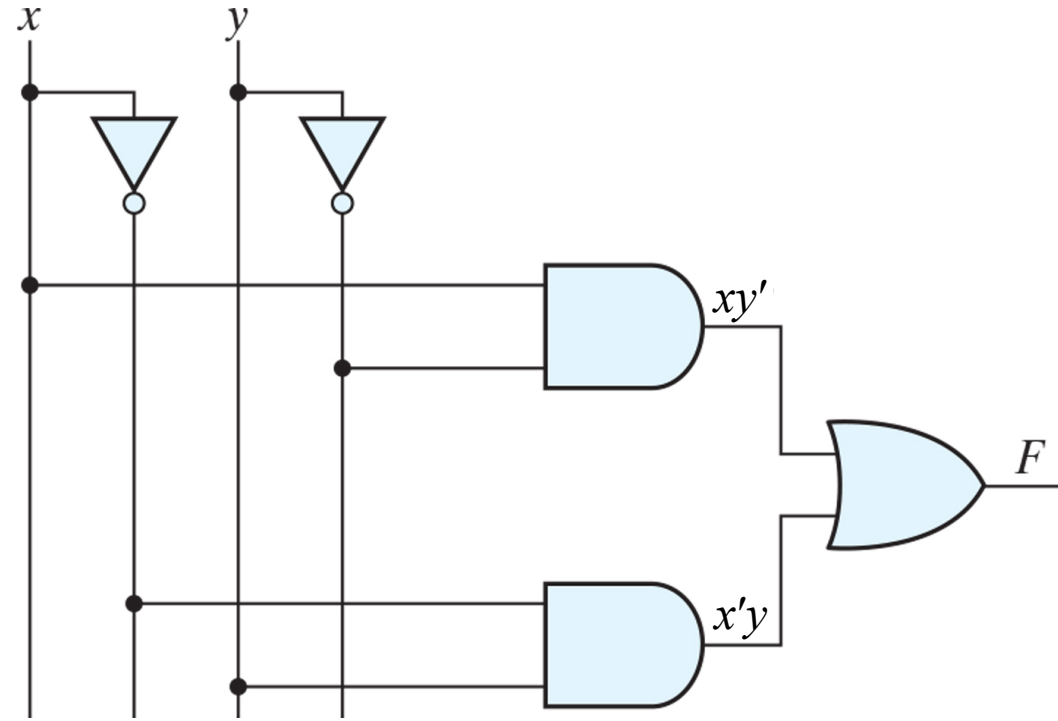
The dual of F_2 is $x + (y' + z')(y + z).$

Complement each literal: $x' + (y + z)(y' + z') = F'_2$

2.5.2 Complement of a Function (4/6)

- **Practice Exercise 2.3** Draw a logic diagram of the Boolean function $F = x'y + xy'$.

Answer:



2.5.2 Complement of a Function (5/6)

- **Practice Exercise 2.4** What Boolean expression is implemented by the following logic diagram?

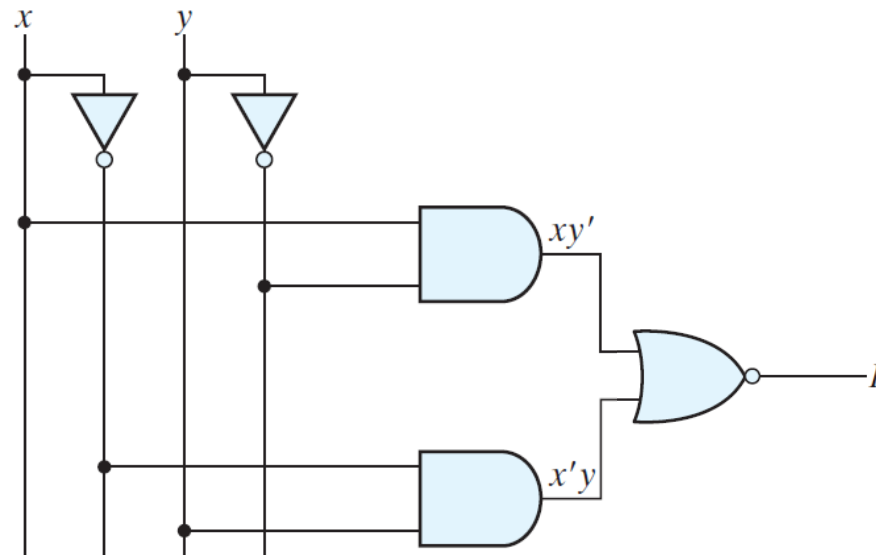


FIGURE PE2.4

Answer: $F = (x'y + xy')' = (x'y)'(xy')' = (x + y')(x' + y)$
 $= xx' + xy + y'x' + yy' = xy + x'y'.$

2.5.2 Complement of a Function (6/6)

- **Practice Exercise 2.5** What truth table is implemented by the logic diagram in Fig. PE2.4?

Answer:

x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

- **Practice Exercise 2.6** Find the complement of the Boolean function $F = A'BC' + A'B'C$.
Answer: $F' = A + BC + B'C$.

2.6 Canonical and Standard Forms (1/12)

- Minterms (最小項、全及項) and Maxterms (最大項、全和項)
 - A minterm: an **AND** term consists of **all literals** in their normal form or in their complement form
 - For example, two binary variables x and y : $xy, xy', x'y, x'y'$
 - It is also called a standard product (標準積)
 - n variables can be combined to form 2^n minterms
 - A maxterm: an **OR** term consists of **all literals** in their normal form or in their complement form
 - For example, two binary variables x and y : $(x' + y'), (x' + y), (x + y'), (x + y)$
 - It is also called a standard sum (標準和)
 - n variables can be combined to form 2^n maxterms
- Each maxterm is the complement of its corresponding minterm, and vice versa.

2.6 Canonical and Standard Forms (2/12)

- In a minterm,
 - primed variable $\leftrightarrow$ “0”
 - unprimed variable $\leftrightarrow$ “1”
- In a maxterm,
 - primed variable $\leftrightarrow$ “1”
 - unprimed variable $\leftrightarrow$ “0”

Table 2.3
Minterms and Maxterms for Three Binary Variables

<i>x</i>	<i>y</i>	<i>z</i>	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

2.6 Canonical and Standard Forms (3/12)

- A Boolean function can be expressed by
 - a truth table
 - sum of minterms that produce a “1”
 - “sum” means the ORing of terms
 - $f_1 = x'y'z + xy'z' + xyz$
 $= m_1 + m_4 + m_7$
 - $f_2 = x'yz + xy'z + xyz' + xyz$
 $= m_3 + m_5 + m_6 + m_7$

Table 2.4
Functions of Three Variables

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.6 Canonical and Standard Forms (4/12)

- The complement of a Boolean function
 - the minterms that produce a “0”
 - $f'_1 = x'y'z' + x'yz' + x'yz + xy'z + xyz' = m_0 + m_2 + m_3 + m_5 + m_6$
 - $f_1 = (f'_1)' = (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z')(x' + y' + z)$
 $= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 \leftarrow$ product of maxterms
 - $f_2 = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$
 $= M_0 \cdot M_1 \cdot M_2 \cdot M_4 \leftarrow$ product of maxterms
- Any Boolean functions expressed as a sum of minterms or a product of maxterms are said to be in *Canonical form* (正規形式).

2.6 Canonical and Standard Forms (5/12)

● **Example 2.4** Express $F = A + B'C$ as a sum of minterms

● Method 1

$$\begin{aligned}
 F &= A + B'C = A(B + B') + B'C = AB + AB' + B'C \\
 &= AB(C + C') + AB'(C + C') + (A + A')B'C \\
 &= ABC + ABC' + AB'C + AB'C' + A'B'C \\
 &= A'B'C + AB'C' + AB'C + ABC' + ABC \\
 &= m_1 + m_4 + m_5 + m_6 + m_7
 \end{aligned}$$

● Method 2

- Built the truth table
 - Insert 1's to F for combinations with $A = 1$ or $BC = 01$
- Brief notation for sum-of-minterms: $F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$
- The symbol Σ stands for the ORing of terms.
 - The numbers are the indices of the minterms of the function.

Table 2.5

Truth Table for $F = A + B'C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

2.6 Canonical and Standard Forms (6/12)

- For n binary variables, there are 2^{2^n} Boolean functions
- **Example 2.5** Express the Boolean function $F = xy + x'z$ as a product of maxterms. Hint: $A + BC = (A + B)(A + C)$
 - $F = xy + x'z = (xy + x')(xy + z) = (x + x')(y + x')(x + z)(y + z)$
 $= (x' + y)(x + z)(y + z)$
 - $x' + y = x' + y + zz' = (x' + y + z)(x' + y + z')$
 - $F = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') = M_0M_2M_4M_5$
 - $F(x, y, z) = \Pi(0, 2, 4, 5)$
 - The product symbol Π denotes the ANDing of maxterms.
 - The numbers are the indices of the maxterms of the function.

2.6 Canonical and Standard Forms (7/12)

- Conversion between Canonical Forms

- $F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$

- $F'(A, B, C) = \Sigma(0, 2, 3) = m_0 + m_1 + m_2$

- By DeMorgan's theorem

$$F = (m_0 + m_2 + m_3)' = m_0' \cdot m_2' \cdot m_3'$$

$$= M_0 M_2 M_3 = \Pi(0, 2, 3)$$

- $m_j' = M_j$

- sum of minterms = product of maxterms

- Interchange the symbols Σ and Π and list those numbers missing from the original form

- Σ of 1's

- Π of 0's

2.6 Canonical and Standard Forms (8/12)

- Example: $F = xy + x'z$
 - $F(x, y, z) = \Sigma(1, 3, 6, 7)$
 - $F(x, y, z) = \Pi(0, 2, 4, 5)$

Table 2.6
Truth Table for $F = xy + x'z$

x	y	z	F	
0	0	0	0	Minterms
0	0	1	1	
0	1	0	0	
0	1	1	1	
1	0	0	0	Maxterms
1	0	1	0	
1	1	0	1	
1	1	1	1	

2.6 Canonical and Standard Forms (9/12)

- **Practice Exercise 2.7** Find a product of maxterms expression for $F(x, y, z) = \Sigma(1, 2, 3, 5, 7)$.
Answer: $F' = \Sigma(0, 4, 6) \rightarrow F(x, y, z) = (x + y + z)(x' + y + z)(x' + y' + z)$
- **Practice Exercise 2.8** Find a sum of minterms expression for $F(x, y, z) = \Pi(1, 3, 4, 6)$.
Answer: $F(x, y, z) = \Sigma(0, 2, 5, 7) = x'y'z' + x'yz' + xy'z + xyz$

- **Practice Exercise 2.9** Identify the minterms and maxterms of the truth table for F shown below.

Answer:

Minterms: $x'y'z, x'yz, xy'z', xyz'$

Maxterms: $(x + y + z), (x + y' + z), (x' + y + z'), (x' + y' + z')$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

2.6 Canonical and Standard Forms (10/12)

- Canonical forms are seldom used due to all variables must be contained in each maxterm or minterm.
- Standard Forms
 - Sum of Products (SPO): $F_1 = y' + xy + x'yz'$
 - Product of Sums (POS): $F_2 = x(y' + z)(x' + y + z')$
 - $F_3 = AB + C(D + E) = AB + CD + CE$

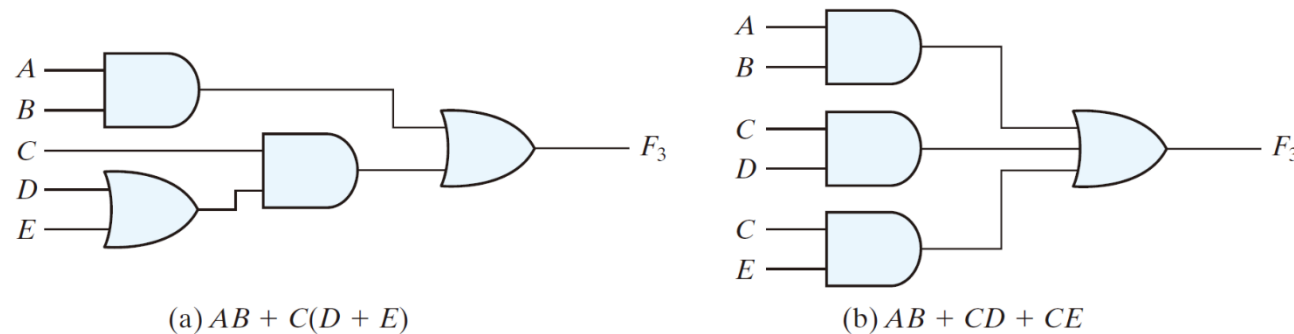


FIGURE 2.4

Three- and two-level implementation

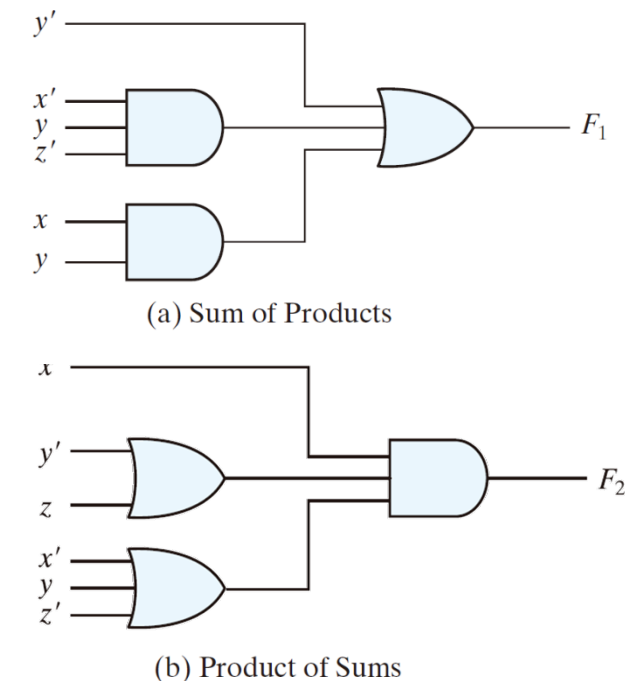


FIGURE 2.3

Two-level implementation

input variables are assumed directly available in their complements.

2.6 Canonical and Standard Forms (11/12)

- **Practice Exercise 2.10** Express the Boolean function $F = A + B'C + AD$ as a sum of minterms.

Answer: $F = \Sigma(2, 3, 8, 9, 10, 11, 12, 13, 14, 15)$

- **Practice Exercise 2.11** Express the Boolean function $F = x'y + xz$ as a product of maxterms.

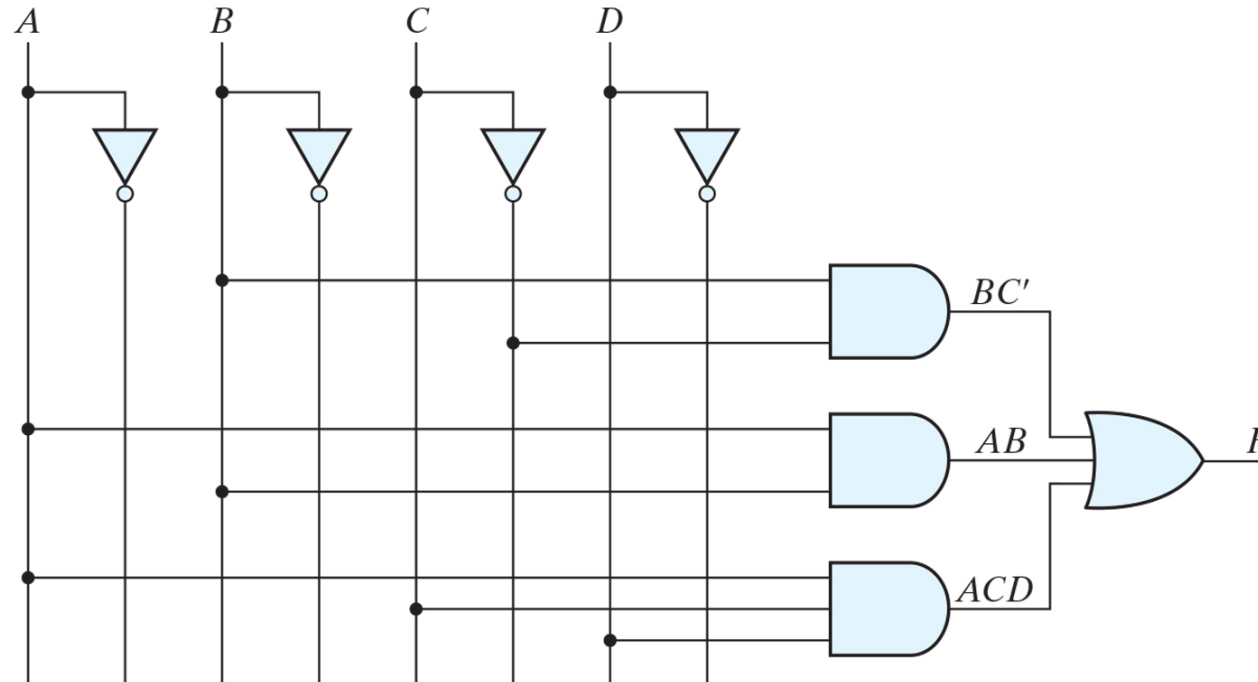
Answer: $F = (x + y + z)(\textcolor{red}{x} + y + z')(x' + y + z)(x' + y' + z)$

typo in the textbook

2.6 Canonical and Standard Forms (12/12)

- **Practice Exercise 2.12** Draw a two-level logic diagram to implement the Boolean function $F = BC' + AB + ACD$

Answer:



2.7 Other Logic Operations (1/2)

- 2^n rows in the truth table of n binary variables
- 2^{2^n} functions for n binary variables
- Example: 16 functions of two binary variables

Table 2.7

Truth Tables for the 16 Functions of Two Binary Variables

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

2.7 Other Logic Operations (2/2)

- All the new symbols except for the exclusive-OR symbol are not in common use by digital designers

Table 2.8
Boolean Expressions for the 16 Functions of Two Variables

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x , but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y , but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y , but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y , then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x , then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

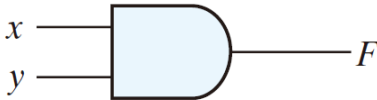
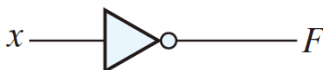
2.8 Digital Logic Gates (1/12)

- Boolean expression: AND, OR, and NOT operations
- Constructing gates of other logic operations
 - the feasibility and economy
 - the possibility of extending the gate's inputs
 - the basic properties of the binary operations
 - the ability of the gate to implement Boolean functions

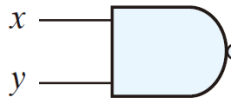
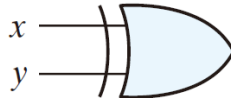
2.8 Digital Logic Gates (2/12)

- Consider the 16 functions
 - two are equal to a constant
 - four are repeated twice
 - inhibition and implication are not commutative or associative
 - the other eight: complement, transfer, AND, OR, NAND, NOR, XOR, and equivalence are used as standard gates
 - complement: inverter
 - transfer: buffer (increasing drive strength)
 - equivalence: XNOR

2.8 Digital Logic Gates (3/12)

Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = x \cdot y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	

2.8 Digital Logic Gates (4/12)

NAND		$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (x + y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

2.8 Digital Logic Gates (5/12)

- Extension to multiple inputs
 - A gate can be extended to multiple inputs
 - if its binary operation is commutative and associative
 - AND and OR are commutative and associative
 - $(x + y) + z = x + (y + z) = x + y + z$
 - $(xy)z = x(yz) = xyz$

2.8 Digital Logic Gates (6/12)

- NAND and NOR are commutative but not associative => they are not extendable

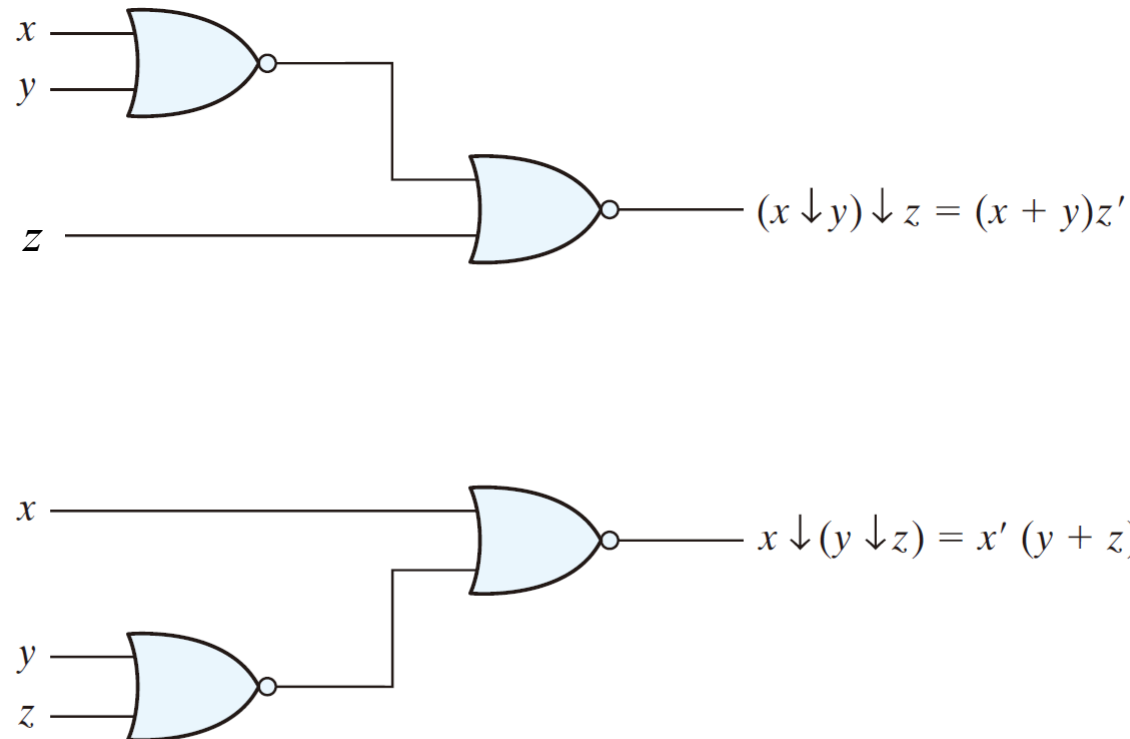


FIGURE 2.6

Demonstrating the nonassociativity of the NOR operator: $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$

2.8 Digital Logic Gates (7/12)

- Multiple-input NOR = a complement of multiple-input OR gate
- Multiple-input NAND = a complement of multiple-input AND
- The cascaded NAND operations = sum of products
- The cascaded NOR operations = product of sums

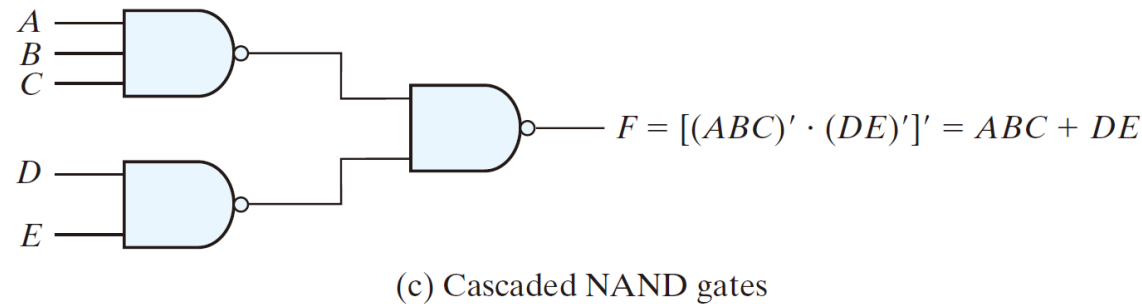
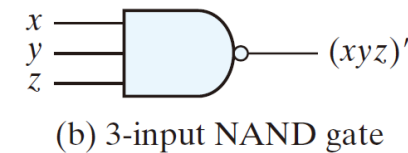
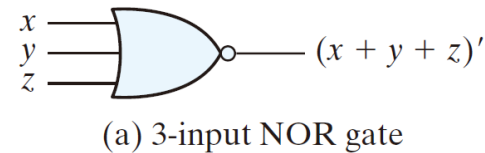
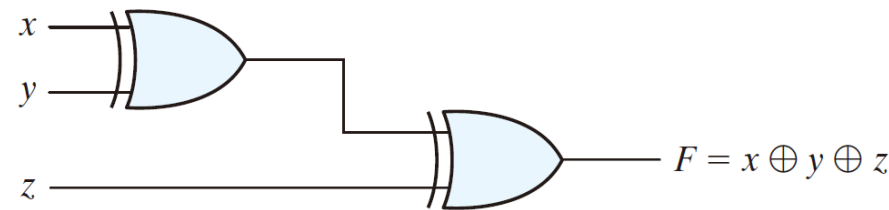


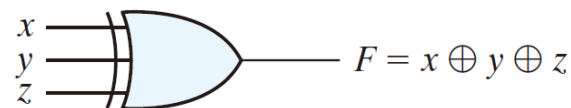
FIGURE 2.7
Multiple-input and cascaded NOR and NAND gates

2.8 Digital Logic Gates (8/12)

- The XOR and XNOR gates are commutative and associative
 - Multiple-input XOR gates are not commonly used.
- XOR is an odd function: it is equal to 1 if the input variables have an odd number of 1's



(a) Using 2-input gates



(b) 3-input gate

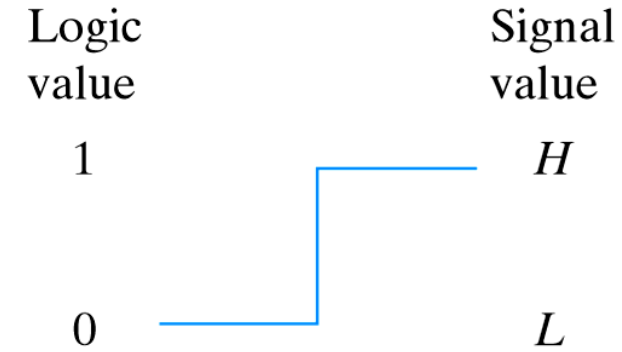
x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(c) Truth table

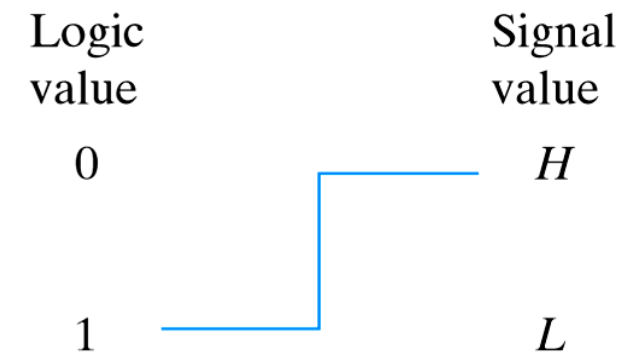
FIGURE 2.8
Three-input exclusive-OR gate

2.8 Digital Logic Gates (9/12)

- Positive and Negative Logic
 - two signal values $\Leftrightarrow$ two logic values
 - positive logic: $H = 1; L = 0$
 - negative logic: $H = 0; L = 1$
- Consider a TTL gate
 - a positive logic AND gate
 - a negative logic OR gate
 - the positive logic is used in this book



(a) Positive logic



(b) Negative logic

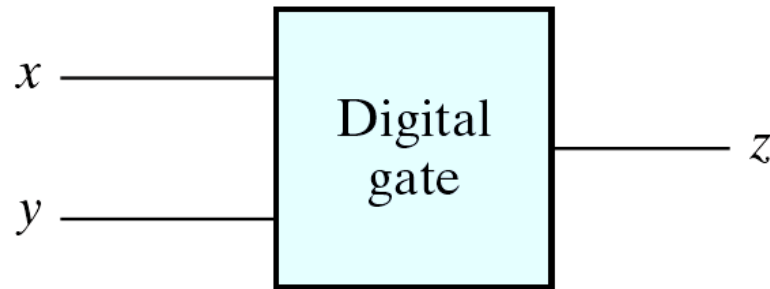
FIGURE 2.9

Signal assignment and logic polarity

2.8 Digital Logic Gates (10/12)

x	y	z
L	L	L
L	H	L
H	L	L
H	H	H

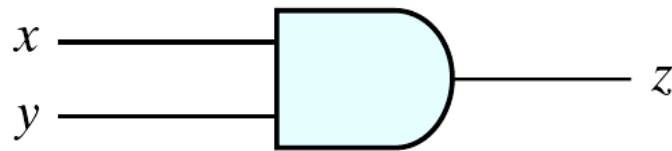
(a) Truth table with H and L



(b) Gate block diagram

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

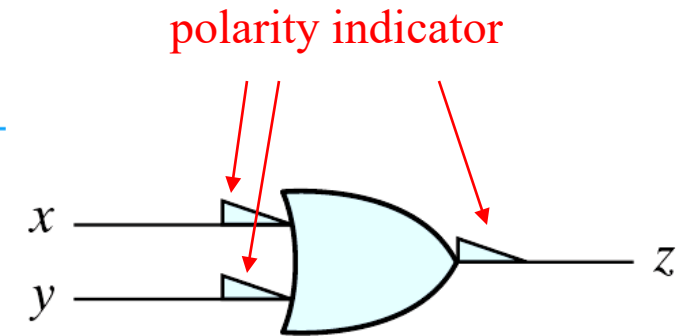
(c) Truth table for positive logic



(d) Positive logic AND gate

x	y	z
1	1	1
1	0	1
0	1	1
0	0	0

(e) Truth table for negative logic



(f) Negative logic OR gate

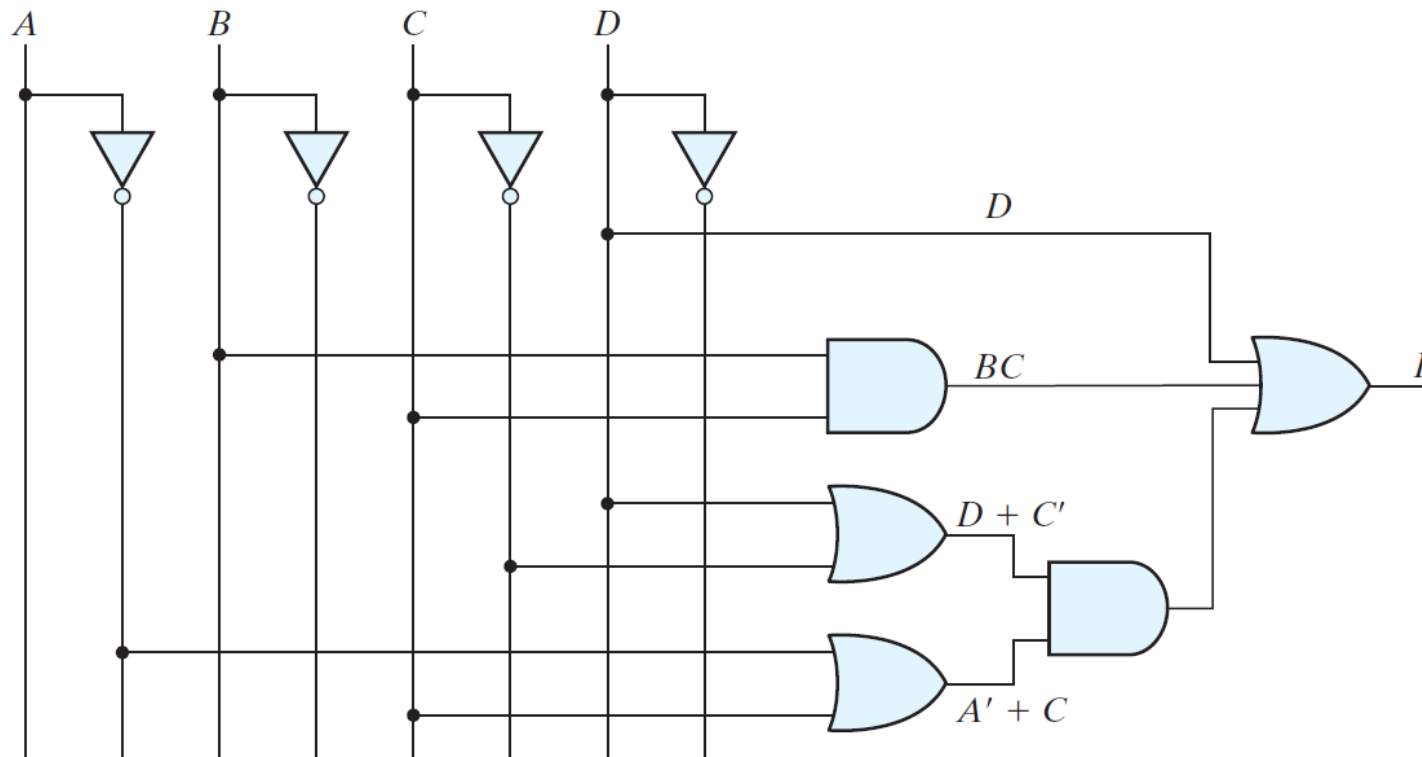
FIGURE 2.10

Demonstration of positive and negative logic

2.8 Digital Logic Gates (11/12)

- **Practice Exercise 2.13** Draw the logic diagram corresponding to the following Boolean expression without simplifying it $F = D + BC' + (D + C')(A' + C)$

Answer:



2.8 Digital Logic Gates (12/12)

- **Practice Exercise 2.14** Implement the Boolean function $F = xz + x'z' + x'y$ with (a) NAND and inverter gates, and (b) NOR and inverter gates

Answer:

(a) NAND gates

$$F = xz + x'z' + x'y$$

$$F' = (xz)' (x'z')' (x'y)'$$

$$F = [(xz)' (x'z')' (x'y)']'$$

(a) NOR gates

$$F' = (x' + z')(x + z)(x + y')$$

$$F = (x' + z')' + (x + z)' + (x + y')'$$

2.9 Integrated Circuits (1/4)

- An integrated circuit (IC) is fabricated on a die of silicon semiconductor crystal called a chip.
- Examples:
 - Small-scale integrated (SSI): < 10 gates
 - Medium-scale integrated (MSI): $10 \sim 100$ gates
 - Large-scale integrated (LSI): $100 \sim \times k$ gates
 - Very large-scale integrated (VLSI): $> \times k$ gates
 - Ultra large-scale integrated (ULASI): $> \times M$ gates

2.9 Integrated Circuits (2/4)

- Digital logic families: circuit technology
 - TTL: transistor-transistor logic
 - ECL: emitter-coupled logic (high speed, high power consumption)
 - MOS: metal-oxide semiconductor (NMOS, high density)
 - CMOS: complementary MOS (low power)

2.9 Integrated Circuits (3/4)

- The characteristics of digital logic families
 - Fan-out (扇出): the number of standard loads that the output of a typical gate can drive
 - Fan-in (扇入): the number of inputs available in a gate
 - Power dissipation (功率消耗): the power consumed by a gate
 - Propagation delay (傳遞延遲): the average transition delay time for the signal to propagate from input to output
 - Noise margin (雜訊邊限): the minimum of external noise voltage that caused an undesirable change in the circuit output

2.9 Integrated Circuits (4/4)

- CAD – Computer-Aided Design
 - Millions of transistors
 - Computer-based representation and aid
 - Automatic the design process
 - Design entry
 - Schematic capture
 - HDL – Hardware Description Language
 - Verilog, VHDL
 - Simulation
 - Physical realization
 - ASIC (Application-Specific IC)
 - FPGA (Field-Programmable Gate Array)
 - PLD (Programmable Logic Device)